

✓ WK6 — Finalizing Models (Hyperparameter Tuning + Ensembles)

Course: AIT-506 Machine Learning | Project: Exam Cheating Detection (5 classes)

This notebook finalizes the two-stage classification approach from WK5 by:

1. Re-extracting EfficientNet-B0 embeddings (the strongest feature source from WK5, which reached macro F1 = 0.90 in the hierarchical setup).
2. Running **GridSearchCV** with **StratifiedKFold** cross-validation for both stages of the model, where the grid also tests **SMOTE** vs class-weighting for handling imbalance. This directly answers the earlier feedback that lower F1 scores suggest too much class imbalance, and that stratified sampling, SMOTE, and grid search should be used to find the best settings for each class.
3. Building a **stacking ensemble** (SVM + Random Forest + Logistic Regression) for each stage, as suggested by the WK6 brief.
4. Comparing the WK5 baseline, the GridSearchCV-tuned model, and the stacking ensemble on the same validation split.

Note on the WK5 baseline number used below (Acc 0.9904 / Macro F1 0.9015): that result came from a hierarchical model in WK5 that used embeddings from a *fine-tuned* EfficientNet-B0. This notebook re-extracts embeddings from a *pretrained-only* (ImageNet) EfficientNet-B0, matching the WK5 backbone shortlisting notebook. The comparison is still meaningful (both are EfficientNet-B0 + two-stage classifier), but keep in mind the feature source is not identical — this is discussed in the report.

✓ 0 Mount Drive, install packages, imports

```
from google.colab import drive
drive.mount('/content/drive')

import subprocess, sys
try:
    import imblearn # noqa
```

```
except ImportError:
    subprocess.check_call([sys.executable, '-m', 'pip', 'install', 'imbalanced-learn', '-q'])
print('imbalanced-learn ready')
```

Mounted at /content/drive
imbalanced-learn ready

```
import hashlib, warnings, time
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from PIL import Image

import torch
import torch.nn as nn
from torchvision import transforms, models

from sklearn.model_selection import train_test_split, GridSearchCV, StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier, StackingClassifier
from sklearn.metrics import (accuracy_score, f1_score, classification_report,
                             confusion_matrix, ConfusionMatrixDisplay)

from imblearn.pipeline import Pipeline as ImbPipeline
from imblearn.over_sampling import SMOTE

warnings.filterwarnings('ignore')
sns.set_theme(style='whitegrid', palette='Set2')
plt.rcParams.update({'figure.dpi': 110, 'axes.titlesize': 13})

SEED = 42
np.random.seed(SEED); torch.manual_seed(SEED)
```

```
DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print('Device:', DEVICE)
```

```
Device: cuda
```

✓ 1 Configuration and stratified train/validation split

Same dataset, classes, and stratified split as WK5 so results are comparable.

```
CANDIDATES = [
    Path('/content/drive/MyDrive/ExamCheatingDataset_preprocessed'),
]
DATA_ROOT = next((p for p in CANDIDATES if p.exists()), None)
assert DATA_ROOT is not None, 'No dataset folder found - edit CANDIDATES.'
TRAIN_DIR = DATA_ROOT / 'train'
IMG_EXTS = {'.jpg', '.jpeg', '.png', '.bmp', '.webp', '.gif'}
CLASSES = sorted(d.name for d in TRAIN_DIR.iterdir() if d.is_dir())
NORMAL_CLS = 'normal act'
SUB_CLASSES = [c for c in CLASSES if c != NORMAL_CLS]
VAL_SPLIT, IMG_SIZE, BATCH = 0.20, 224, 64
print('Classes:', CLASSES)
```

```
Classes: ['direct cheating', 'giving code', 'giving object', 'looking friend', 'normal act']
```

```
def list_images(folder):
    return [p for p in folder.rglob('*') if p.suffix.lower() in IMG_EXTS and p.is_file()]

seen, paths, labels, dropped = set(), [], [], 0
for c in CLASSES:
    for p in list_images(TRAIN_DIR / c):
        try:
            h = hashlib.md5(p.read_bytes()).hexdigest()
        except Exception:
            continue
        if h in seen:
```

```

        dropped += 1; continue
    seen.add(h); paths.append(str(p)); labels.append(CLASSES.index(c))

paths, labels = np.array(paths), np.array(labels)
tr_paths, va_paths, y_train, y_val = train_test_split(
    paths, labels, test_size=VAL_SPLIT, stratify=labels, random_state=SEED)
print(f'Kept {len(paths)} images ({dropped} duplicates removed) | '
      f'train {len(tr_paths)} | val {len(va_paths)}')

```

Kept 1551 images (11 duplicates removed) | train 1240 | val 311

✓ 2 EfficientNet-B0 feature extraction

EfficientNet-B0 was the strongest single backbone in WK5 (best accuracy per parameter). We strip its classifier head and use it as a fixed feature extractor (1280-dim embeddings per image), then standardise the features.

```

IMAGENET_TF = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225]),
])

effnet = models.efficientnet_b0(weights=models.EfficientNet_B0_Weights.DEFAULT)
effnet.classifier = nn.Identity()
effnet = effnet.to(DEVICE).eval()
for p in effnet.parameters():
    p.requires_grad_(False)

@torch.no_grad()
def extract_features(path_list):
    feats = []
    for i in range(0, len(path_list), BATCH):
        imgs = torch.stack([IMAGENET_TF(Image.open(p).convert('RGB'))
                             for p in path_list[i:i + BATCH]]).to(DEVICE)
        feats.append(effnet(imgs).cpu().numpy())

```

```

return np.vstack(feats)

t0 = time.time()
Xtr_raw = extract_features(tr_paths)
Xva_raw = extract_features(va_paths)
print(f'Embeddings extracted in {time.time() - t0:.0f}s | shape {Xtr_raw.shape}')

scaler = StandardScaler().fit(Xtr_raw)
Xtr = scaler.transform(Xtr_raw)
Xva = scaler.transform(Xva_raw)

```

Downloading: "https://download.pytorch.org/models/efficientnet_b0_rwightman-7f5810bc.pth" to /root/.cache/torch
 100%|██████████| 20.5M/20.5M [00:00<00:00, 146MB/s]
 Embeddings extracted in 15s | shape (1240, 1280)

✓ 3 Stage labels

- **Stage 1** — binary: normal act (0) vs any cheating-related behaviour (1)
- **Stage 2** — 4-class subtype, fit only on images that are cheating-related

`giving object` (15) and `direct cheating` (12) are very rare classes — this is the imbalance the GridSearchCV step below is designed to address.

```

normal_idx      = CLASSES.index(NORMAL_CLS)
sub_classes_idx = [CLASSES.index(c) for c in SUB_CLASSES]
sub_pos         = {CLASSES.index(c): i for i, c in enumerate(SUB_CLASSES)}

# Stage 1 labels
yb_train = (y_train != normal_idx).astype(int)
yb_val   = (y_val   != normal_idx).astype(int)

# Stage 2 labels (cheating-related images only)
cheat_mask_tr = (y_train != normal_idx)
cheat_mask_va = (y_val   != normal_idx)

```

```
Xsub_tr = Xtr[cheat_mask_tr]
ysub_tr = np.array([sub_pos[l] for l in y_train[cheat_mask_tr]])
Xsub_va = Xva[cheat_mask_va]
ysub_va = np.array([sub_pos[l] for l in y_val[cheat_mask_va]])

print('Stage 1 train class balance (normal / cheating):', np.bincount(yb_train))
print('Stage 2 train class balance:', np.bincount(ysub_tr), '->', SUB_CLASSES)

Stage 1 train class balance (normal / cheating): [504 736]
Stage 2 train class balance: [ 10 215  12 499] -> ['direct cheating', 'giving code', 'giving object', 'looking
```

✓ 4 GridSearchCV — Stage 1 (normal vs cheating)

5-fold **StratifiedKFold** cross-validation. The grid searches over:

- **Sampling strategy:** no resampling (relying on `class_weight='balanced'`) vs **SMOTE** oversampling of the minority class before fitting
- **Classifier family:** SVM (RBF) vs Random Forest
- Classifier-specific hyperparameters (`C`, `gamma`, `n_estimators`, `max_depth`)

Scoring is **macro F1**, matching the project's primary metric.

```
cv1 = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)

pipe1 = ImbPipeline([
    ('sampler', 'passthrough'),
    ('clf', SVC()),
])

param_grid1 = [
    {
        'sampler': ['passthrough', SMOTE(random_state=SEED)],
        'clf': [SVC(probability=True, class_weight='balanced', random_state=SEED)],
        'clf__C': [1, 10, 100],
```

```

        'clf__gamma': ['scale', 'auto'],
        'clf__kernel': ['rbf'],
    },
    {
        'sampler': ['passthrough', SMOTE(random_state=SEED)],
        'clf': [RandomForestClassifier(class_weight='balanced', random_state=SEED, n_jobs=-1)],
        'clf__n_estimators': [200, 400],
        'clf__max_depth': [None, 20],
    },
]

```

```

grid_s1 = GridSearchCV(pipe1, param_grid1, scoring='f1_macro', cv=cv1, n_jobs=-1, refit=True)
grid_s1.fit(Xtr, yb_train)

```

```

print('Best Stage 1 params:', grid_s1.best_params_)
print(f'Best Stage 1 CV macro F1: {grid_s1.best_score_:.4f}')

```

```

Best Stage 1 params: {'clf': SVC(class_weight='balanced', probability=True, random_state=42), 'clf__C': 10, '
Best Stage 1 CV macro F1: 0.9707

```

✓ 5 GridSearchCV — Stage 2 (4-class subtype)

Same idea, but with **3-fold** StratifiedKFold because the rarest subtype (**direct cheating**, 12 images total) only has ~9-10 training images — a 5-fold split would leave too few samples per fold. SMOTE's **k_neighbors** is also reduced to 3 for the same reason.

```

cv2 = StratifiedKFold(n_splits=3, shuffle=True, random_state=SEED)

```

```

pipe2 = ImbPipeline([
    ('sampler', 'passthrough'),
    ('clf', SVC()),
])

```

```

param_grid2 = [

```

```

{
    'sampler': ['passthrough', SMOTE(random_state=SEED, k_neighbors=3)],
    'clf': [SVC(probability=True, class_weight='balanced', random_state=SEED)],
    'clf__C': [1, 10, 100],
    'clf__gamma': ['scale', 'auto'],
    'clf__kernel': ['rbf'],
},
{
    'sampler': ['passthrough', SMOTE(random_state=SEED, k_neighbors=3)],
    'clf': [RandomForestClassifier(class_weight='balanced', random_state=SEED, n_jobs=-1)],
    'clf__n_estimators': [200, 400],
    'clf__max_depth': [None, 20],
},
]

```

```

grid_s2 = GridSearchCV(pipe2, param_grid2, scoring='f1_macro', cv=cv2, n_jobs=-1, refit=True)
grid_s2.fit(Xsub_tr, ysub_tr)

```

```

print('Best Stage 2 params:', grid_s2.best_params_)
print(f'Best Stage 2 CV macro F1: {grid_s2.best_score_:.4f}')

```

```

Best Stage 2 params: {'clf': RandomForestClassifier(class_weight='balanced', n_jobs=-1, random_state=42), 'cl
Best Stage 2 CV macro F1: 0.5665

```

✓ 6 End-to-end evaluation — GridSearchCV-tuned two-stage model

Combine the best Stage 1 and Stage 2 estimators and evaluate on the held-out validation split, exactly as in WK5.

```

def two_stage_predict(s1_model, s2_model, X):
    s1_pred = s1_model.predict(X)
    final = np.full(len(X), normal_idx, dtype=int)
    cheat = (s1_pred == 1)
    if cheat.any():
        s2_pred = s2_model.predict(X[cheat])

```



```

        final[np.where(cheat)[0]] = [sub_classes_idx[i] for i in s2_pred]
    return final

tuned_pred = two_stage_predict(grid_s1.best_estimator_, grid_s2.best_estimator_, Xva)
tuned_acc = accuracy_score(y_val, tuned_pred)
tuned_f1 = f1_score(y_val, tuned_pred, average='macro', zero_division=0)

print(f'GridSearchCV-tuned two-stage - Accuracy: {tuned_acc:.4f} | Macro F1: {tuned_f1:.4f}')
print(classification_report(y_val, tuned_pred, target_names=CLASSES, digits=3))

```

```

GridSearchCV-tuned two-stage - Accuracy: 0.9678 | Macro F1: 0.7558
      precision    recall  f1-score   support

direct cheating      0.000      0.000      0.000         2
   giving code      0.981      0.981      0.981        54
   giving object      0.750      1.000      0.857         3
   looking friend      0.953      0.976      0.964       125
      normal act      0.984      0.969      0.976       127

   accuracy                   0.968       311
   macro avg      0.734      0.785      0.756       311
   weighted avg      0.963      0.968      0.965       311

```

✓ 7 Stacking ensemble

Combine three different model families — SVM, Random Forest, and Logistic Regression — using a `StackingClassifier`. Each base model votes via its predicted probabilities, and a Logistic Regression meta-model learns how to combine those votes. This is built separately for Stage 1 and Stage 2.

```

def make_stack(cv_splits):
    estimators = [
        ('svm', SVC(kernel='rbf', C=10, gamma='scale', probability=True,
                    class_weight='balanced', random_state=SEED)),
        ('rf', RandomForestClassifier(n_estimators=300, class_weight='balanced',

```

```

        random_state=SEED, n_jobs=-1)),
        ('lr', LogisticRegression(max_iter=1000, class_weight='balanced')),
    ]
    return StackingClassifier(
        estimators=estimators,
        final_estimator=LogisticRegression(max_iter=1000, class_weight='balanced'),
        stack_method='predict_proba', cv=cv_splits, n_jobs=-1
    )

stack_s1 = make_stack(cv1)
stack_s1.fit(Xtr, yb_train)

stack_s2 = make_stack(cv2)
stack_s2.fit(Xsub_tr, ysub_tr)

stack_pred = two_stage_predict(stack_s1, stack_s2, Xva)
stack_acc = accuracy_score(y_val, stack_pred)
stack_f1 = f1_score(y_val, stack_pred, average='macro', zero_division=0)

print(f'Stacking ensemble two-stage – Accuracy: {stack_acc:.4f} | Macro F1: {stack_f1:.4f}')
print(classification_report(y_val, stack_pred, target_names=CLASSES, digits=3))

```

Stacking ensemble two-stage – Accuracy: 0.9678 | Macro F1: 0.8502

	precision	recall	f1-score	support
direct cheating	0.400	1.000	0.571	2
giving code	0.981	0.981	0.981	54
giving object	0.600	1.000	0.750	3
looking friend	0.992	0.960	0.976	125
normal act	0.976	0.969	0.972	127
accuracy			0.968	311
macro avg	0.790	0.982	0.850	311
weighted avg	0.976	0.968	0.971	311

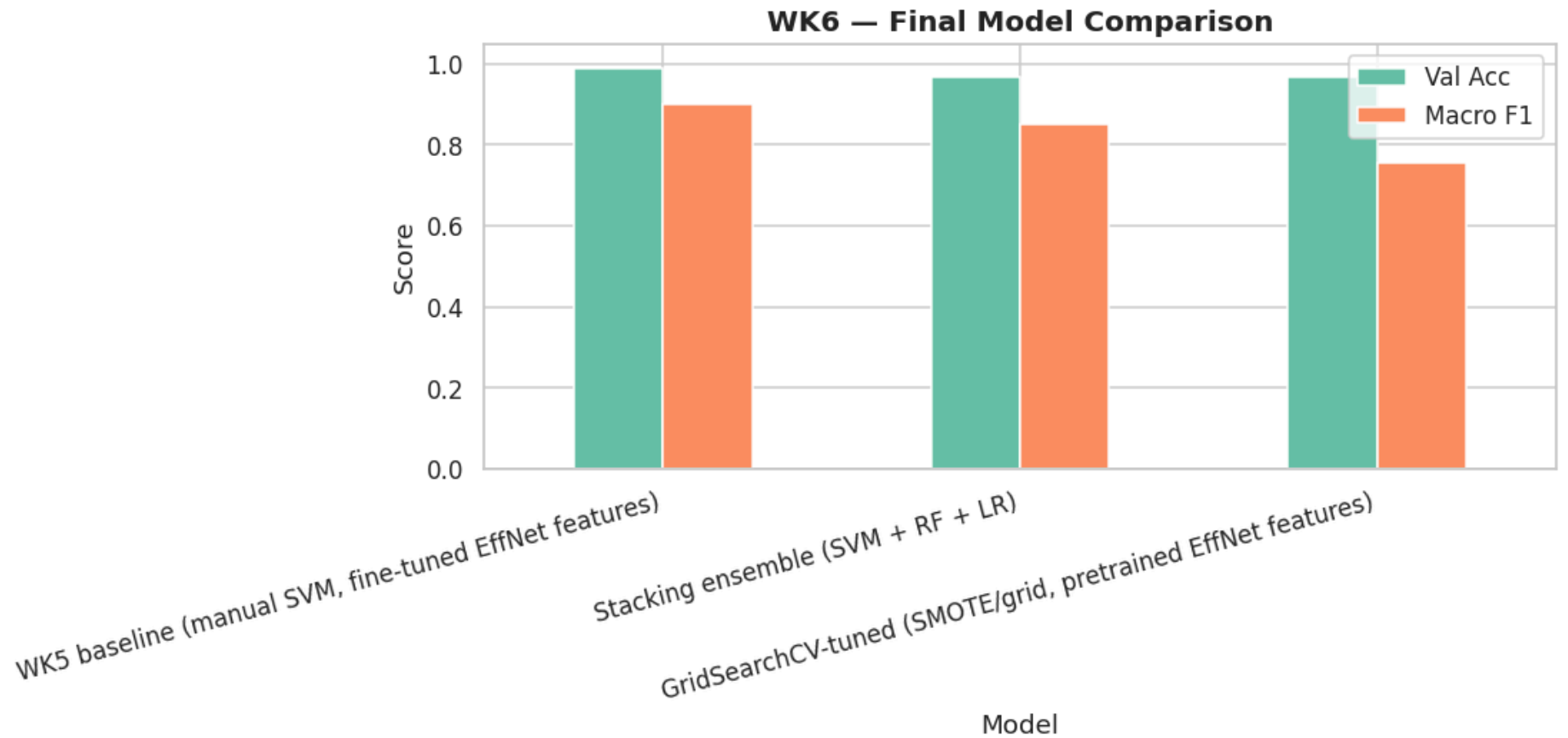
✓ 8 Final comparison — WK5 baseline vs GridSearchCV-tuned vs Stacking ensemble

```
results_table = pd.DataFrame([
    {'Model': 'WK5 baseline (manual SVM, fine-tuned EffNet features)', 'Val Acc': 0.9904, 'Macro F1': 0.9015},
    {'Model': 'GridSearchCV-tuned (SMOTE/grid, pretrained EffNet features)', 'Val Acc': tuned_acc, 'Macro F1': tuned_f1},
    {'Model': 'Stacking ensemble (SVM + RF + LR)', 'Val Acc': stack_acc, 'Macro F1': stack_f1},
])
results_table = results_table.sort_values('Macro F1', ascending=False).reset_index(drop=True)
results_table.index += 1
print('=' * 70)
print(' WK6 FINAL MODEL COMPARISON')
print('=' * 70)
print(results_table.round(4).to_string())
print('=' * 70)

ax = results_table.set_index('Model')[['Val Acc', 'Macro F1']].plot(
    kind='bar', figsize=(10, 5), color=sns.color_palette('Set2', 2))
ax.set_title('WK6 – Final Model Comparison', fontweight='bold')
ax.set_ylabel('Score'); ax.set_ylim(0, 1.05)
ax.set_xticklabels(results_table['Model'], rotation=15, ha='right')
plt.tight_layout()
plt.savefig('wk6_model_comparison.png', bbox_inches='tight')
plt.show()
```

WK6 FINAL MODEL COMPARISON

	Model	Val Acc	Macro F1
1	WK5 baseline (manual SVM, fine-tuned EffNet features)	0.9904	0.9015
2	Stacking ensemble (SVM + RF + LR)	0.9678	0.8502
3	GridSearchCV-tuned (SMOTE/grid, pretrained EffNet features)	0.9678	0.7558



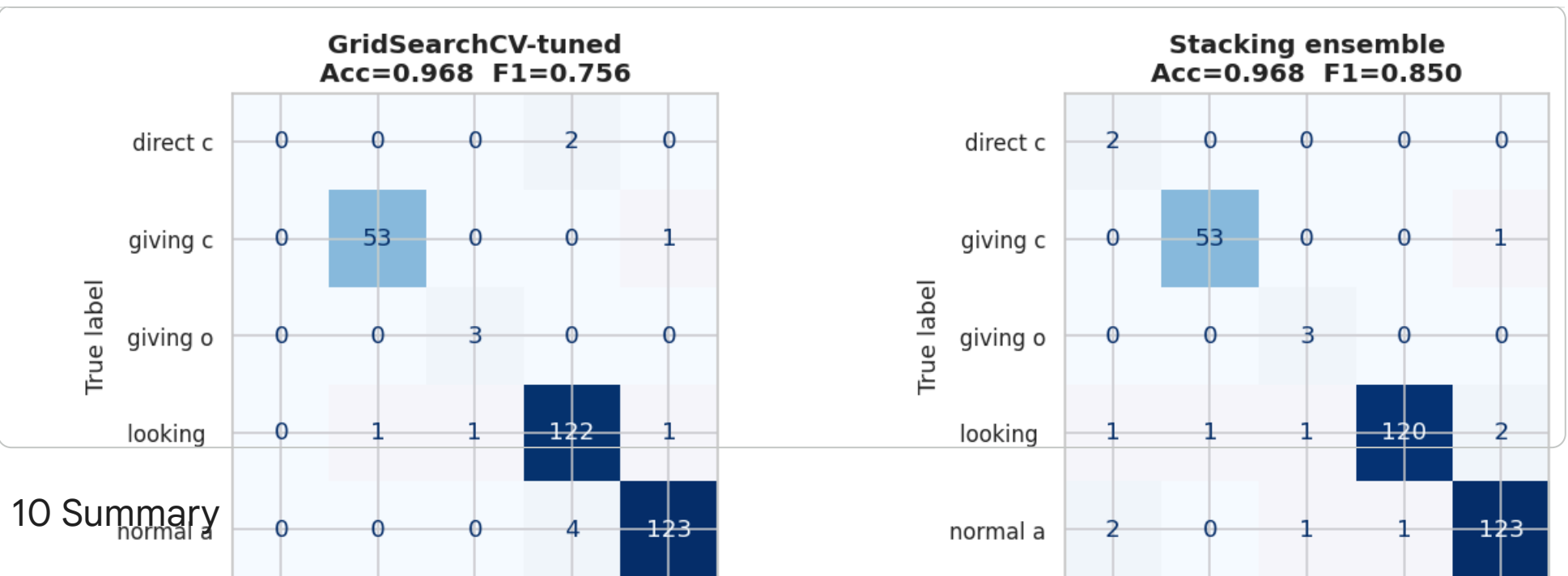
✓ 9 Confusion matrices — tuned model vs stacking ensemble

```
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

cm_tuned = confusion_matrix(y_val, tuned_pred)
ConfusionMatrixDisplay(cm_tuned, display_labels=[c[:8] for c in CLASSES]).plot(
    ax=axes[0], colorbar=False, cmap='Blues')
axes[0].set_title(f'GridSearchCV-tuned\nAcc={tuned_acc:.3f}  F1={tuned_f1:.3f}',
                  fontweight='bold')
axes[0].tick_params(axis='x', rotation=30)

cm_stack = confusion_matrix(y_val, stack_pred)
ConfusionMatrixDisplay(cm_stack, display_labels=[c[:8] for c in CLASSES]).plot(
    ax=axes[1], colorbar=False, cmap='Blues')
axes[1].set_title(f'Stacking ensemble\nAcc={stack_acc:.3f}  F1={stack_f1:.3f}',
                  fontweight='bold')
axes[1].tick_params(axis='x', rotation=30)

plt.tight_layout()
plt.savefig('wk6_confusion_matrices.png', bbox_inches='tight')
plt.show()
```



10 Summary

- The GridSearchCV step searched over SMOTE vs class-weighting, SVM vs Random Forest, and their key hyperparameters, using stratified cross-validation — directly addressing the earlier feedback about imbalance.
- The stacking ensemble combines three different model types so that their errors are less correlated, which can help most on the rarest classes (`direct cheating`, `giving object`).
- Compare the three rows in the table above on **Macro F1** (primary metric, treats every class equally) and **Val Acc**. Note which configuration won for Stage 1 and Stage 2 (`grid_s1.best_params_`, `grid_s2.best_params_`) — this tells you whether SMOTE or class-weighting, and which classifier, worked best for this dataset.
- Once this notebook has been run, paste the printed numbers (`grid_s1.best_params_`, `grid_s2.best_params_`, `tuned_acc`, `tuned_f1`, `stack_acc`, `stack_f1`, and the classification reports) back so the WK6 APA 7 report can be written up with your actual results.